

Conv-ChArT — the target and the loss, spelled out

Every symbol, every constant, every operation. Constants read from configs/rev640.yaml at build time.

Contents

0. What problem the loss is solving	1
1. The target: turning corner coordinates into an image	2
1.1 What we start with	2
1.2 The Gaussian blob (<code>_splat_max</code>)	2
1.3 Forcing the peak to exactly 1.0 — and what <code>rint</code> is	3
1.4 The class map: one more coordinate change	4
1.5 Worked example, one corner	4
2. The loss	4
2.1 From logit to probability, and why the network outputs a logit	4
2.2 Plain BCE: the baseline that everything else modifies	5
2.3 The focal term α : down-weight what is already right	5
2.4 The penalty-reduction term β : forgive near-misses	6
2.5 Normalisation: dividing by N	6
2.6 The refiner’s loss	7
3. Every symbol, in one table	7
4. Every operation, in one table	8
5. The three-line summary	8

0. What problem the loss is solving

The detector outputs two images of numbers. Training has to tell each number “you should have been higher” or “you should have been lower”. That is all a loss function does.

The two outputs:

1. **The heatmap**, one number per input pixel: *is there a corner here?* Shape $480 \times 640 = \mathbf{307,200}$ numbers.
2. **The class map**, 16 numbers per H/4 cell: *if there is a corner here, which of the board’s 16 corners is it?* Shape $16 \times 120 \times 160 = \mathbf{307,200}$ numbers.

The board is 5×5 squares, so it has $(5-1) \times (5-1) = 16$ **interior corners** — points where four squares meet. Those are what we detect. Corners on the outer boundary are not used: they touch only two squares, so they are ordinary edge points, not the X-shaped junctions the network is trained to find.

The central difficulty, as one number. A frame has at most 16 corners among 307,200 pixels. That is **0.0052%** of the image. If you simply told the network “minimise your average error”, it would answer “*there is never a corner anywhere*”, be right 99.9948% of the time, and be useless. Everything that looks strange in the loss below exists to defeat that one degenerate answer.

1. The target: turning corner coordinates into an image

1.1 What we start with

The synthetic generator knows exactly where every corner is, because it *placed* the board. Per frame, per corner, it gives us:

symbol	what it is	type	where it comes from
$p = (p_x, p_y)$	corner position in the input image	float , sub-pixel	<code>record["corners"][i]["x"], ["y"]</code> — the placement homography in <code>dcc/synth.py</code>
v	is this corner visible?	bool	<code>["visible"]</code> — false if off-frame or hidden behind an occluder
k	which corner is it?	int, 0..15	<code>["index"]</code> — fixed board-intrinsic numbering, row-major over the interior grid

p being **float** is the reason the rest of this section is fiddly. A corner does not land on a pixel; it lands *between* pixels, at e.g. $x = 431.27$. The target is an array with integer indices. Bridging that gap is what 1.3 and 1.4 are about.

1.2 The Gaussian blob (`_splat_max`)

For each visible corner we paint a blob into the target. From `dcc/targets.py`:

$$Y(i, j) = \exp \left(\frac{-[(i - p_x)^2 + (j - p_y)^2]}{2\sigma^2} \right)$$

Reading it left to right:

- i, j are **integer pixel coordinates** — where we are in the target array.
- p_x, p_y are the **float** corner position. So $(i - p_x)$ is the horizontal distance from this pixel to the true corner, and it is generally not a whole number.
- $(i - p_x)^2 + (j - p_y)^2$ is squared Euclidean distance — Pythagoras. No square root is taken, because the formula wants d^2 anyway.
- $\exp(-\cdot)$ turns “distance” into “closeness”. Distance 0 gives $e^0 = 1$; the value falls smoothly toward 0 as you move away.
- σ (**sigma**) sets *how fast* it falls. Larger σ = wider, fatter blob.

This is an *unnormalised* Gaussian. A textbook Gaussian probability density carries a $1/(2\pi\sigma^2)$ factor so its total volume is 1. We deliberately omit it, because we want the **peak value to be exactly 1.0**, not the volume. The target is “confidence per pixel”, not a probability distribution over pixels.

The values of σ used in this project:

where	symbol	value	units	source
heatmap	σ_{hm}	2.0	input pixels	<code>sigma_hm</code> in <code>configs/rev640.yaml</code>
class map	σ_{cls}	1.0	H/4 cells (4 input px each)	<code>sigma_cls</code>
refiner	σ_{ref}	1.5	1/8-pixel grid units	<code>render_refiner_target</code> default

The $\pm 3\sigma$ window

`_splat_max` does not evaluate the Gaussian over the whole 480×640 image. It only touches a box $\pm 3\sigma$ around the corner:

```
x0 = floor(cx - 3*sigma)      x1 = ceil(cx + 3*sigma)
```

At 3σ the Gaussian equals $e^{-4.5} = 0.01111$ — about 1% of peak — and beyond that it is numerically irrelevant. This is a **pure speed optimisation**: at $\sigma = 2.0$ it evaluates a 14×14 box instead of 307,200 pixels, per corner.

- **floor** rounds *down* to the nearest integer: `floor(4.7) = 4`, `floor(-0.3) = -1`. Used for the lower bound so the box never starts *inside* the region we want.
- **ceil** rounds *up*: `ceil(4.2) = 5`. Used for the upper bound, same reason. Together they guarantee the box fully covers the $\pm 3\sigma$ region.
- **max(0, ...)** and **min(w-1, ...)** clip the box to the array bounds, so a corner near the image edge writes a partial blob instead of crashing on a negative index.

Why *max*-combine, not *sum* — hence the name `_splat_max`

Two corners can be close enough that their blobs overlap. The function combines them with `np.maximum` (elementwise “keep whichever is larger”), **not** addition:

$$Y(i, j) = \max_c \exp \left(\frac{-\|(i, j) - p^{(c)}\|^2}{2\sigma^2} \right)$$

taken over all visible corners c . If we summed instead, a pixel midway between two corners could exceed **1.0**, and the loss would be asking a sigmoid to output more than 1 — impossible, so that pixel would emit permanent unsatisfiable gradient. Max keeps every value inside $[0, 1]$ by construction. (This is CornerNet’s convention, stated explicitly in their paper.)

Overlap is the normal case, not an edge case. At our smallest trained scale, $s = 12$ px between adjacent corners, the midpoint sits 6.0 px from each corner, where each blob is still 0.011.

1.3 Forcing the peak to exactly 1.0 — and what `rint` is

After all blobs are painted, `render_heatmap` runs a **second loop** that overwrites one pixel per corner:

```
jx, jy = int(np rint(px)), int(np rint(py))
hm[jy, jx] = 1.0
```

rint means “round to nearest integer” — the name comes from the C library function `rint`, “round to integer”. `rint(431.27) = 431`; `rint(431.8) = 432`. It picks the pixel whose centre is closest to the true corner. NumPy’s `rint` breaks exact .5 ties toward the *even* integer (banker’s rounding), so `rint(2.5) = 2` but `rint(3.5) = 4`. That never matters in practice, but it is why the function is `rint` rather than `round`.

Why this line must exist. The Gaussian evaluated at a pixel *centre* equals 1.0 only if the corner sits exactly on that centre — a measure-zero coincidence that essentially never happens. Without this line the largest target value in a real frame would be something like 0.9692, never exactly 1.0.

That matters because the loss defines its positives as `y == 1.0`, an **exact float equality test**. With no forced peak, no pixel is ever positive, the positive branch of the loss never fires, and the network is only ever told to output zeros. **The forced peak is what makes the “there IS a corner here” signal exist at all.**

The cost, stated honestly. The blob is centred on the true float position p , but the 1.0 is written at `rint(p)`. Those differ by up to **0.5 px per axis**, i.e. 0.707 px diagonally. So the target is slightly self-inconsistent: its declared peak is not quite at the top of its own blob. That is tolerable because the heatmap only has to identify the right *pixel* — sub-pixel precision is the refiner’s job, and the refiner measures exactly this leftover offset.

1.4 The class map: one more coordinate change

The class head predicts at **H/4 resolution** — one cell per 4×4 block of input pixels, so 120×160 cells. Identity is a regional property (which marker lies next to this corner), so it does not need full resolution, and 1/16 of the cells is 1/16 of the compute.

Converting an input-pixel position to a cell position is where a subtle bug lives, so the code is explicit:

$$x_c = \frac{x + 0.5}{4} - 0.5$$

Why not just $x/4$? Because of the **pixel-centre convention**: pixel index x means “the centre of pixel x ”, which occupies the real interval $[x - 0.5, x + 0.5]$. Cell 0 covers input pixels 0,1,2,3 — the real interval $[-0.5, 3.5]$ — whose centre is at $x = 1.5$. A correct mapping must therefore send $x = 1.5$ to $x_c = 0$:

- ours: $(1.5 + 0.5)/4 - 0.5 = 0$. Correct.
- naive $x/4$: $1.5/4 = 0.375$. **Wrong by 0.375 cells = 1.5 input pixels.**

That 1.5-px bias would be invisible during training (target and readout would share it) but would corrupt every identity lookup at inference. The same convention is used by `dcc.pipeline`’s `grid_sample` readout, which is why the two agree.

For the forced peak the class map uses **floor**, not **rint**:

```
jx = int(np.floor((px + 0.5) / 4))
```

That is the *containing* cell — which 4×4 block the corner actually falls inside — which is the meaningful notion for a cell. **rint** would instead give the *nearest cell centre*, a different cell whenever the corner sits in the outer half of its block.

Channel k . Each of the 16 corners gets its **own channel**: corner k ’s blob is painted only into `ct[k]`. The 16 channels are therefore mutually exclusive by construction, and the network learns “this location is corner #7”, not merely “this location is a corner”.

1.5 Worked example, one corner

Corner $k = 5$ at $p = (431.27, 208.63)$, visible, $\sigma_{hm} = 2.0$:

step	operation	result
1	$\pm 3\sigma$ box	$x \in [425, 438], y \in [202, 215]$
2	Gaussian at pixel (431, 209)	$d^2 = 0.27^2 + 0.37^2$, so $Y = 0.9741$
3	Gaussian at pixel (433, 209)	$d^2 = 1.73^2 + 0.37^2$, so $Y = 0.6762$
4	rint	<code>(rint(431.27), rint(208.63)) = (431, 209)</code>
5	force peak	<code>hm[209, 431] = 1.0</code> , overwriting the 0.9741 from step 2
6	class-map position	$x_c = (431.27 + 0.5)/4 - 0.5 = 107.4425$
7	class containing cell	<code>floor((431.27+0.5)/4) = 107</code> , so <code>ct[5, 52, 107] = 1.0</code>

2. The loss

2.1 From logit to probability, and why the network outputs a logit

The final layer outputs a raw real number per pixel called a **logit**, z . It can be anything from $-\infty$ to $+\infty$. The **sigmoid** squashes it into a probability:

$$\hat{p} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

$z = 0$ gives $\hat{p} = 0.5$; $z = +5$ gives 0.9933; $z = -5$ gives 0.0067.

A symbol clash worth flagging, because it is genuinely confusing: σ means the Gaussian width in section 1 and the sigmoid function in section 2. They are unrelated. Both are standard notation in their respective literatures, so we keep both and warn rather than invent new symbols.

Why emit z instead of $\hat{p}$ directly? Numerical stability. Computing $\log \hat{p}$ from a stored $\hat{p}$ loses precision when $\hat{p}$ is tiny — and in a map that is 99.9948% background, most $\hat{p}$ are tiny. In logit space, $\log \sigma(z)$ is computed by `F.logsigmoid(z)` in one numerically-stable step. This is load-bearing rather than pedantic: training runs in **bfloat16**, which has roughly 3 decimal digits of precision, and the naive form produces NaN near the forced peaks.

Bias initialisation. The final conv layer’s bias starts at **-2.19** (`dcc/model.py:289`), so before learning anything the network outputs $\hat{p} = \sigma(-2.19) = 0.101$ everywhere — a deliberate “probably no corner here” prior. Without it an untrained net says 0.5 at all 307,200 pixels, the first gradient step is dominated by 307,200 confidently-wrong background pixels, and training begins by violently collapsing everything to zero. Starting near the true base rate skips that phase.

2.2 Plain BCE: the baseline that everything else modifies

Binary cross-entropy for one pixel with a binary target $t \in \{0, 1\}$:

$$L_{BCE} = -[t \log \hat{p} + (1 - t) \log(1 - \hat{p})]$$

Only one term survives per pixel. If $t = 1$ it is $-\log \hat{p}$, which grows without bound as $\hat{p} \rightarrow 0$ — “you missed a corner”. If $t = 0$ it is $-\log(1 - \hat{p})$ — “you invented a corner”. The $-\log$ is small when the prediction is right and large when it is confidently wrong.

BCE alone fails here, measurably. On one frame with 16 corners predicted at $\hat{p} = 0.7$ and background at $\hat{p} = 0.01$, the background contributes **99.8% of the total loss**: 307,184 pixels each contributing a little swamp 16 pixels each contributing a lot. The gradient is dominated by pixels that are *already correct*.

2.3 The focal term α : down-weight what is already right

Multiply each pixel’s BCE by a factor that vanishes once the prediction is good:

$$L = -[t(1 - \hat{p})^\alpha \log \hat{p} + (1 - t)\hat{p}^\alpha \log(1 - \hat{p})]$$

- α (**alpha**) = **2** (alpha in `configs/rev640.yaml`). It is called γ in the original focal-loss paper (Lin et al., RetinaNet); we call it α following CornerNet. Same quantity.
- $(1 - \hat{p})^\alpha$ multiplies positives: if the network already says $\hat{p} = 0.99$ at a real corner, this factor is 1e-04 and that pixel is effectively removed from the loss.
- $\hat{p}^\alpha$ multiplies negatives: if it already says 0.01 at background, same factor, same removal.

It is symmetric across classes. This is the most commonly misunderstood point, so concretely, at $\alpha = 2$:

case	$\hat{p}$	BCE	with focal	suppression
easy negative	0.01	0.01005	0.0000010	10,000 ×
easy positive	0.99	0.01005	0.0000010	10,000 ×
hard positive	0.10	2.30259	1.8650939	1 ×
hard negative	0.90	2.30259	1.8650939	1 ×

α does **not** weight positives above negatives. It weights **hard above easy**, identically in both classes. The imbalance is fixed as a *side effect*: background is both numerous and easy, so background is exactly what gets suppressed. On the frame from 2.2 the background’s share of the loss falls from **99.8% to 37.5%**.

A per-class constant weight (`pos_weight` in PyTorch’s BCE) is a different mechanism: it scales all positives alike, and cannot shift attention toward the *remaining* hard cases as training progresses.

2.4 The penalty-reduction term β : forgive near-misses

Everything above used a binary t . Now the graded Gaussian Y finally enters. For one pixel:

$$L = - \begin{cases} (1 - \hat{p})^\alpha \log \hat{p} & \text{if } Y = 1 \\ (1 - Y)^\beta \hat{p}^\alpha \log(1 - \hat{p}) & \text{otherwise} \end{cases}$$

and the total loss sums this over every pixel.

- β (**beta**) = 4 (beta in `configs/rev640.yaml1`, overridable per run as `focal_beta`).
- **The positive branch is selected by $Y = 1$ exactly** — the forced-peak pixel from 1.3 and nothing else. In code that is `pos = y == 1.0`.
- $(1 - Y)^\beta$ multiplies the negative branch only. Y near 1 (just beside a corner) makes it tiny, so the pixel is barely penalised. $Y = 0$ (far background) makes it 1, so the penalty is full.

Critically, Y is a *weight*, not a target value. For every negative pixel the loss is monotone in $\hat{p}$: its minimum is always at $\hat{p} = 0$, never at $\hat{p} = Y$. A pixel with $Y = 0.61$ is not being asked to output 0.61 — it is being asked to output 0, just very gently. (Contrast MSE-regression-to-a-Gaussian, as used by stacked-hourglass pose estimators, where Y genuinely *is* the value to match.)

The forgiveness profile at our settings ($\sigma_{hm} = 2.0$, $\beta = 4$):

distance d from corner (px)	Y	$(1 - Y)^\beta$	% of full penalty
0	1.0000	0.00e+00	0.000%
1	0.8825	1.91e-04	0.019%
2	0.6065	2.40e-02	2.397%
3	0.3247	2.08e-01	20.802%
4	0.1353	5.59e-01	55.897%
5	0.0439	8.35e-01	83.550%
6	0.0111	9.56e-01	95.630%

At $d = 1$ px a pixel keeps **0.019%** of its penalty. Put plainly: lighting up the pixel *next to* the corner is essentially free. The **half-forgiveness radius**, where the penalty reaches 50%, is **3.83 px**. That is the number worth carrying: at these settings the loss barely polices a 3.8-pixel neighbourhood around every corner, which is why measured peaks come out broad.

How σ and β divide the work. Two stages, one knob each:

1. distance $\rightarrow Y$, controlled by **** σ alone****. Only the ratio d/σ matters, so σ purely rescales the distance axis.
2. $Y \rightarrow$ penalty weight, controlled by **** β alone****. It is a gamma curve on $(1 - Y)$: at $\beta = 1$ forgiveness *equals* the Gaussian exactly; $\beta > 1$ bulges it outward; $\beta = 0$ deletes it.

$\beta = 0$ deserves an explicit note because it is not intuitive. $(1 - Y)^0 = 1$ for every pixel, so **the graded target is ignored entirely** and Y survives only as the binary mask $Y = 1$. So $\beta = 0$ is *standard focal loss on a one-hot target* — and still **not** BCE, because α remains.

2.5 Normalisation: dividing by N

$$L_{\text{detector}} = \frac{L_{hm} + \lambda_{cls} L_{cls}}{\max(N, 1)}$$

- N = **total visible corners in the whole batch** — `nvis.sum()` in `tools/train_detector.py:589`, built from `int(vis.sum())` per sample in `dcc/dataset.py:33`.

- Both terms are **sum**-reduced, not mean-reduced, then divided by N once. The loss is therefore “average cost per visible corner”, so a frame showing 3 corners contributes proportionally less than one showing 16. Mean-reduction would divide by pixel count instead and make every frame contribute equally regardless of how much board it contains.
- **** max($N, 1$) prevents division by zero.**** Negative frames (no board) and fully-occluded frames have $N = 0$; without the clamp the loss is $0/0 = \text{NaN}$ and a single such sample destroys the run. With it, an all-background frame gives a small finite loss — which is correct, since “predict nothing here” is genuinely what we want from it.
- **The same N divides both heads.** The class map has 16 channels and so holds roughly $16 \times$ more positives than the heatmap; a per-head normaliser would inflate L_{cls} by about $16 \times$ and let identity dominate localisation.
- $\lambda_{cls} = \mathbf{1.0}$ (`lambda_cls`) is the relative weight of identity against localisation. It is 1.0 because the two heads have near-equal element counts (307,200 vs 307,200, within 0%), so no rebalancing is needed.

2.6 The refiner’s loss

The refiner sees a 24×24 crop and outputs a 64×64 map covering the central 8×8 pixels at $8 \times$ resolution — so one output cell is $1/8$ of an input pixel. Its target comes from `render_refiner_target`:

$$u^* = 31.5 + 8d_x, \quad v^* = 31.5 + 8d_y$$

- $d = (d_x, d_y)$ is the **sub-pixel offset** between the crop’s integer centre and the true corner — precisely the leftover that the heatmap’s `rint` discarded.
- The **8** converts input pixels into output cells ($8 \times$ upsampling).
- The **31.5** is the centre of a 64-wide axis under the pixel-centre convention: indices 0..63 span $[-0.5, 63.5]$, whose midpoint is 31.5. So $d = 0$ lands exactly at the centre.
- Support is ± 3.9375 px, asserted in the code: $31.5 - 8 \times 3.9375 = 0$, so any larger offset would fall outside the map.

Same `_splat_max`, same forced `rint` peak, same `focal()`. Only the normaliser differs — divide by **batch size** rather than corner count, because each crop contains exactly one corner by construction.

3. Every symbol, in one table

symbol	name	value	units	where it comes from
p_x, p_y	corner position	continuous	input px	synth placement homography
v	visibility flag	bool	—	<code>record["corners"][i]["visible"]</code>
k	corner index	0..15	—	board-intrinsic numbering
Y	target value	$[0, 1]$	—	<code>dcc/targets.py</code>
σ_{hm}	heatmap Gaussian width	2.0	input px	<code>sigma_hm</code>
σ_{cls}	class Gaussian width	1.0	H/4 cells	<code>sigma_cls</code>
σ_{ref}	refiner Gaussian width	1.5	$1/8$ px	code default
z	logit	any real	—	network output
$\hat{p}$	predicted probability	$[0, 1]$	—	$\sigma(z)$
α	focal exponent	2	—	alpha ; γ in Lin et al.
β	penalty-reduction exponent	4	—	beta / <code>focal_beta</code>

symbol	name	value	units	where it comes from
N	visible corners per batch	0.. $B \times 16$	—	<code>nvis.sum()</code>
λ_{cls}	class-head weight	1.0	—	<code>lambda_cls</code>
bias init	final-conv bias	-2.19	logit	<code>dcc/model.py:289</code> , gives $\hat{p}_0 = 0.101$
τ_{hm}	peak threshold (<i>inference</i>)	0.3	—	<code>tau_hm</code> — not in the loss
τ_{id}	identity threshold (<i>inference</i>)	0.5	—	<code>tau_id</code> — not in the loss

τ_{hm} and τ_{id} are listed only to head off a natural confusion. They are **decode-time** thresholds, used by `dcc/pipeline.py` to turn a predicted map into detections. They play no part in training and appear nowhere in the loss.

4. Every operation, in one table

operation	what it does	why it is there
$\exp(-d^2/2\sigma^2)$	distance $\rightarrow$ closeness in $(0, 1]$	smooth target, graded partial credit
<code>np.maximum</code>	elementwise larger-of-two	overlapping blobs must not exceed 1.0
<code>floor</code>	round down	lower bound of the $\pm 3\sigma$ box; <i>containing</i> cell for the class map
<code>ceil</code>	round up	upper bound of the box
<code>rint</code>	round to <i>nearest</i> integer	picks the pixel forced to 1.0, so <code>y == 1.0</code> can ever be true
<code>max(0,...), min(w-1,...)</code>	clip indices	edge corners write partial blobs instead of crashing
$\sigma(z) = 1/(1 + e^{-z})$	logit $\rightarrow$ probability	bounds the output to $[0, 1]$
<code>F.logsigmoid(z)</code>	$\log \sigma(z)$, stably	naive $\log \hat{p}$ gives NaN in bfloat16
$(1 - \hat{p})^\alpha, \hat{p}^\alpha$	down-weight easy examples	defeats the 0.0052%-positive imbalance
$(1 - Y)^\beta$	discount near-miss negatives	graded forgiveness; the only place Y 's <i>value</i> is read
<code>y == 1.0</code>	exact float equality	selects the positive branch; works <i>only</i> because of <code>rint</code>
<code>.sum()</code> then $/N$	per-corner normalisation	frames with few corners contribute less
<code>max(N, 1)</code>	clamp the divisor	negative frames would otherwise give $0/0 = \text{NaN}$

5. The three-line summary

1. **The target** paints an unnormalised Gaussian bump at every visible corner, combines overlaps by max so nothing exceeds 1, and force-writes exactly 1.0 at the nearest pixel (`rint`) so the loss has something to call positive.
2. **The loss** is cross-entropy times two correction factors: α suppresses already-correct pixels of *both* classes, so 307,200 background pixels cannot drown 16 corners; and β discounts the penalty on pixels *near* a corner in proportion to the Gaussian value there.
3. **The normaliser** makes the loss “cost per visible corner”, shares one N across both heads, and clamps it so empty frames cannot produce NaN.